

Lecture 7: Basic text analysis

Alex Reinhart
Machine Learning II: 46-927

March 10, 2021

Announcements

- ▶ Project reports due tomorrow night
- ▶ Presentations do not need to be turned in until Tuesday morning, so we have them in case of Zoom issues during the presentations
- ▶ Remember to check the report and presentation rubrics on Canvas—they explain what I'm expecting
- ▶ And remember your presentation must be ≤ 8 minutes!

Today:

- ▶ Some discussion of text data
- ▶ Latent Dirichlet Allocation

Text data

This relates to our earlier discussion of similarity, and more generally to thinking about the structure of data.

It will give us a nice chance to illustrate several ideas, as well as introduce an important topic.

That said: We are using text data as an example, but not diving deeply into it. Most of the things we're going to talk about are very simplistic. Realistic modeling of text is an entire field.

Check out `http:`

`//www.cs.cmu.edu/afs/cs/user/tbergkir/www/11711fa17/`

Text data

Let's think a bit about the special case of text data.

Text data can be incredibly rich in information

- ▶ Corporate reports
- ▶ Tweets
- ▶ Web sites
- ▶ Medical records
- ▶ News stories, investor calls, etc.

We would like to be able to include text data in our analyses.

However, it is complicated by the fact that interesting text data is often *unstructured*.

We need to find representations of the data that will allow us to extract useful information or features for our models.

Representing text data

Throughout this discussion, assume that we have a collection of documents, $i = 1, \dots, n$. Each document is just a collection of free text.

Our first step in analyzing text data is finding a useful **representation** for the documents. We want our representation to:

- ▶ Be easy to generate from the raw documents, and be easy to work with
- ▶ Highlight important aspects of the documents, and suppress unimportant aspects

There is kind of a **trade-off** between these two ideas. Having someone read every document and categorize them by meaning might highlight the important aspects, but it certainly wouldn't be easy.

Bag-of-words representation

Bag-of-words representation of a document is very simple-minded: just list all the words and how many times they appeared. E.g.,

will = 21; thank = 18; great = 16; hillary = 15; crooked = 11;
make = 5; ...

Very easy to generate and easy to use (first idea), but is it too much of a reduction, or can it still be useful (second idea)?

Idea: by itself “great” can take on a lot of meanings, but we can learn from the **other words** in the document besides “great”. E.g.,

- ▶ Words: “will”, “thank”, “hillary”, “make”, “crooked” suggest one context for the word “great”.
- ▶ Words: “Some”, “born” “achieve”, “greatness”, “thrust”, “upon”, “them” suggest another context.

Counting words: Term-document matrix

Bag-of-words turns every document into a vector in $\mathbb{R}^W$

Counting words:

- ▶ First make a list of all of the words present in the documents
- ▶ Index the words $w = 1, \dots, W$ (e.g., in alphabetical order), and the documents $d = 1, \dots, D$ (just pick some order)
- ▶ For each document d , count how many times each word w appears (could be zero), and call this X_{dw} . The vector $X_d = (X_{d1}, \dots, X_{dW})$ gives us the **word counts** for the d th document
- ▶ Do the same thing for the query: let Y_w be the number of times the w th word appears, so the vector $Y = (Y_1, \dots, Y_W)$ contains the word counts for the query

Simple example

Documents:

document 1: "We estimated the parameter from data."

document 2: "We learned the model from data with our 'Machine'."

$D = 2$ documents, $W = 11$ words. For each document, we count the number of occurrences of each word.

Term-document matrix:

	we	estimated	the	model	from	machine	data
Doc 1	1	1	1	0	1	0	1
Doc 2	1	0	1	1	1	1	1

Now what? Thinking about our representation

This already gives us a convenient vector representation for each document ($X_d \in \mathbb{R}^W$).

We might want to:

- ▶ Use these as features in prediction/classification algorithms.
- ▶ Use these to cluster documents into groups.
- ▶ Use these to search for documents similar to a query or example document.

Is this the best form for these features?

Is there a good way to convert this to similarities/distances?

Distances and similarity measures

We can also define a measure of **dissimilarity** or **distance** between documents. What if we just use a measure of distance between the vectors?

Start with the usual measures of distance between n -dimensional vectors X, Y :

- ▶ The ℓ_2 or **Euclidean** distance is (tends to work better here)

$$\|X - Y\|_2 = \sqrt{\sum_{i=1}^n (X_i - Y_i)^2}$$

- ▶ The ℓ_1 or **Manhattan** distance is

$$\|X - Y\|_1 = \sum_{i=1}^n |X_i - Y_i|$$

Varying document lengths and normalization

Different documents have different lengths. Total word counts:

doc 1	doc 2	doc 3	doc 4	doc 5	doc 6	doc 7	doc 8
3114	1976	3330	2143	8962	6524	4618	1766

Wikipedia entry on Michelangelo the painter is almost twice as long as that on Michelangelo the TMNT (6524 vs 3330 words).

We should **normalize** the count vectors X_d in some way

Varying document lengths and normalization

- ▶ **Document length** normalization: divide X by its sum,

$$X \leftarrow X / \sum_{w=1}^W X_w$$

(Like the probability of picking each word, if you sampled a word at random from the document.)

- ▶ **ℓ_2 length** normalization: divide X by its ℓ_2 length,

$$X \leftarrow X / \|X\|_2$$

Common words

So far we've dealt with varying document lengths. What about some words being more helpful than others? **Common words**, especially, are not going to help us find relevant documents

To deal with common words, we could just keep a list of words like “the”, “this”, “that”, etc. to exclude from our representation. These are called “stopwords”. But this would be both **too crude** and **time consuming**

In particular, the frequency and importance of words depends on context!

Common words and IDF weighting

Inverse document frequency (IDF) weighting is smarter and more efficient than a fixed list.

- ▶ For each word w , let n_w be the number of documents that contain this word
- ▶ Let D be the total number of documents
- ▶ Then for each vector X_d , multiply w th component by $\log(D/n_w)$

If a word appears in every document, then it gets a weight of zero, so effectively tossed out of the representation

(IDF performs something like simplistic **variable selection**)

Putting it all together

Think of the document-term matrix:

	word 1	word 2	...	word W
doc 1				
doc 2				
$\vdots$				
doc D				

- ▶ **Normalization** scales each *row* by something (divides a row vector X by its sum $\sum_{i=1}^W X_i$ or its ℓ_2 norm $\|X\|_2$)
- ▶ **IDF weighting** scales each *column* by something (multiplies the w th column by $\log(D/n_w)$)
- ▶ We can use both, just normalize first and then perform IDF weighting

Stemming

Having words “connect”, “connects”, “connected” “connecting”, “connection”, etc. in our representation is extraneous. **Stemming** reduces all of these to the single stem word “connect”

Can a simple list of rules provide perfect stemming? **It seems not:** consider “connect” and “connectivity”, but “relate” and “relativity”; or “sand” and “sander”, but “wand” and “wander”.

Can be much worse in other languages...

Now what?

We have a representation of documents as bag-of-words vectors, and we have a well-designed distance between them. What can we do with distances?

- ▶ Retrieve documents similar to a query or to another document
- ▶ Clustering! (K -medoids, hierarchical clustering)
- ▶ Multi-dimensional scaling! (Even isomap)
- ▶ Classification (given some labeled examples)

We can also use the document-term matrix as an X matrix of features into many of the algorithms we've learned.

Not just bag-of-words

Bag-of-words is the simplest approach to representing text data. It is not a good representation of what exist in Natural Language Processing (NLP). This is a massive branch of ML.

Some extensions and alternatives:

- ▶ n-grams (combinations of words)
- ▶ Models with memory (like HMMs)
- ▶ Models that incorporate parts of speech
- ▶ word2vec (a particular vector representation of words)

Check out [http:](http://www.cs.cmu.edu/afs/cs/user/tbergkir/www/11711fa17/)

[//www.cs.cmu.edu/afs/cs/user/tbergkir/www/11711fa17/](http://www.cs.cmu.edu/afs/cs/user/tbergkir/www/11711fa17/)

Latent Dirichlet Allocation (LDA)

Let's think about one model for representing text data

Important (and silly): We've recently learned about Linear Discriminant Analysis (LDA). We're now learning about Latent Dirichlet Allocation (LDA). These are absolutely unrelated topics.

If you talk to a statistician, LDA tends to refer to the former. If you talk to a machine learning person, it is slightly more likely to refer to the latter.

Latent Dirichlet Allocation (LDA)

Topic modeling describes a set of methods which focus on automatically identifying global themes or topics in a set of documents. **Latent Dirichlet Allocation (LDA)** is one of the simplest and most famous approaches.

The collection of documents is assumed to have a set of latent factors (topics). Each document has a certain amount of each factor, and those factors determine the actual words used in the document.

Latent Dirichlet Allocation (LDA)

For example: An article about computational biology might discuss both biological topics and statistical topics. As the biological content goes up, it's more likely that biology-related words appear. As the statistical content goes up, it's more likely that statistical words appear.

It's a very simple model, but it has been shown to work quite well. It also admits many extensions and improvements.

It should also remind you a lot of both factor models and mixture models!

LDA document generation process

LDA pretends that the words were generated according to the following process:

1. For each topic, generate a distribution over words

$$\beta_k \sim \text{Dirichlet}(\eta)$$

2. Randomly select a distribution of topics for the document

$$\theta_d \sim \text{Dirichlet}(\alpha)$$

3. For each word:

- 3.1 Randomly choose a topic from your topic distribution

$$z_{di} \sim \text{Multinomial}(\theta_d)$$

- 3.2 Randomly choose a word according to the distribution for this topic

$$w_{ij} \sim \text{Multinomial}(\beta_{z_{di}})$$

LDA document generation process

This allows:

- ▶ Documents to have multiple topics.
- ▶ Groups of words to be related to one another by their topic.

LDA: the model

$$p(\beta_{1:K}, \theta_{1:D}, z_{1:D}, w_{1:D}) = \prod_{i=1}^K p(\beta_i) \prod_{d=1}^D p(\theta_d) \left(\prod_{n=1}^N p(z_{d,n} | \theta_d) p(w_{d,n} | \beta_{1:K}, z_{d,n}) \right)$$

- ▶ $\beta_{1:K}$ distributions over words for each topic
- ▶ $\theta_{1:D}$ Proportions of each topic in the d^{th} document
- ▶ $z_{d,n}$, the topic assignment of the n^{th} word in the d^{th} document
- ▶ $w_{d,n}$ the actual n^{th} word in the d^{th} document

We do inference on the parameters of this model conditional on the observed $w_{d,n}$. Bayesian methods are used to do this approximately and quickly.

LDA: output

Once we have estimated the parameter of this model, we now have

- ▶ Estimates of the topic proportions for each document
- ▶ Estimates of the word distribution for each topic

We can use these to model the data set and the different kinds of documents within it. We can even use them as features in more complicated ML methods!

LDA: connection to NMF

LDA: not just for documents

Suppose I have a bunch of individuals and I want to understand their behavior. At p time points, they choose whether or not to take actions.

I could imagine this behavior being driven by a number of factors, the relative importance of each being different for each individual. This could be modeled by LDA or similar models!